

DOCUMENTO DE ARQUITETURA — V1.1

Chat Service

Serviço de mensageria em tempo real — standalone, genérico e integrável por qualquer sistema via contratos bem definidos.

Autor Emanuel Martins Data Maio 2026 Status Planejamento

1 Contexto e Motivação

O objetivo deste serviço é construir uma **base de chat reutilizável**: um núcleo pequeno e autossuficiente que pode ser hospedado e integrado em qualquer projeto que precise de mensageria em tempo real. Não é dedicado a nenhum sistema específico — expõe contratos estáveis e qualquer integrador os consome.

● Atômico

Funciona de forma standalone, sem depender de nenhum outro serviço. Sobe sozinho com Docker.

● Integrável

Expõe contratos bem definidos (gRPC + eventos RabbitMQ) para ser consumido por qualquer projeto.

● Reutilizável

Base de chat para qualquer aplicação futura — sempre que um projeto precisar de chat, esse é o serviço.

● Nível de mercado

Arquitetura de gateway com multiplexing, sequence numbers, delivery guarantees e presence tracking.

2 Análise de Protocolos

WebSocket "pelado" com JSON é suficiente tecnicamente, mas não demonstra engenharia de sistemas. O que diferencia uma plataforma de mercado é o que acontece **sobre** o transporte.

XMPP

Base: WhatsApp foi construído em cima. Federado, maduro, extensível via XEPs.

Decisão: XML verboso, ecossistema parado desde 2015. Não agrega ao portfólio.

Matrix

Base: Protocolo moderno, federado, E2E encryption nativo. Usado por governos e NASA.

Decisão: Implementar homeserver do zero = 6+ meses. Usar SDK existente elimina o aprendizado.

MTPROTO

Base: Protocolo próprio do Telegram. Binário, eficiente, criptografia customizada.

Decisão: Proprietário e complexo demais. Fora de escopo.

Discord Gateway

Base: WebSocket multiplexado + Protobuf + heartbeats + sequence numbers. Erlang/OTP backend.

Decisão: [Referência principal](#). Implementável com Phoenix Channels.

Phoenix Channels

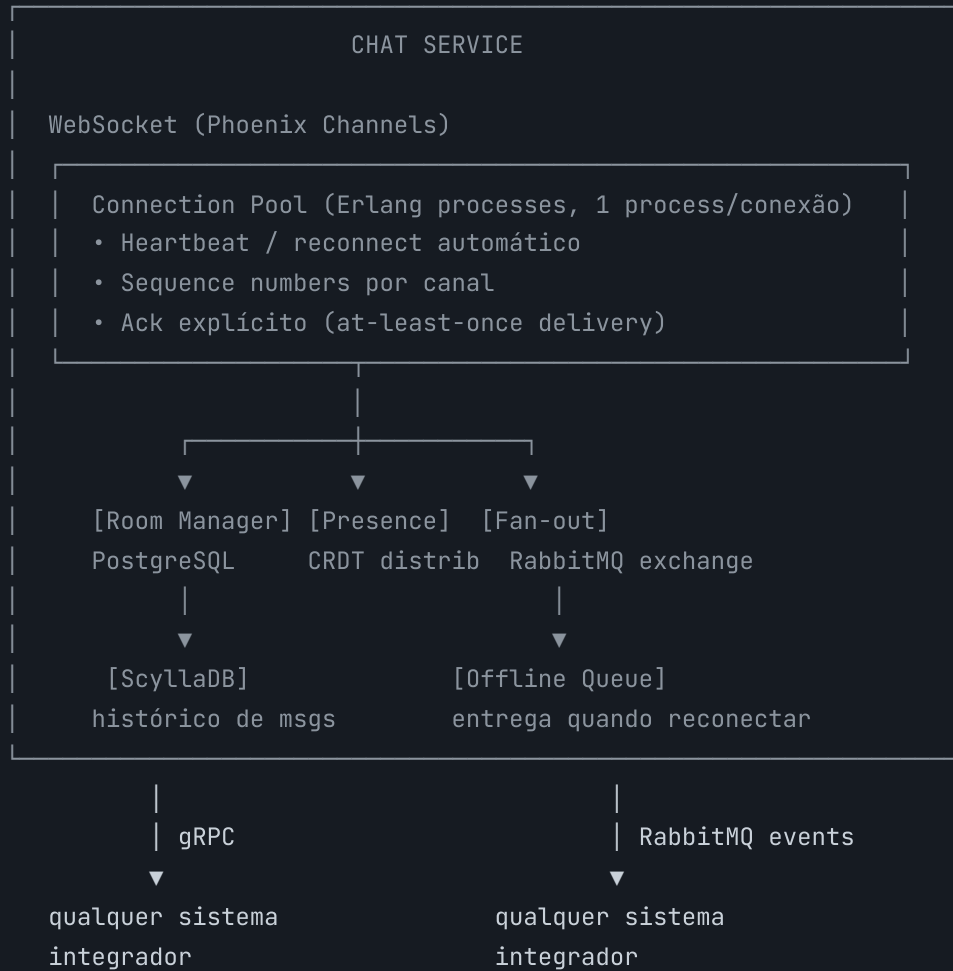
Base: O que inspirou o modelo do Discord. Multiplexing nativo, Presence como CRDT distribuído.

Decisão: [Stack escolhida](#). O BEAM é a arquitetura — presença CRDT, isolamento de conexão e tolerância a falha são propriedades estruturais do runtime.

O que torna uma arquitetura de chat "nível de mercado" não é o protocolo de transporte — é a **engenharia sobre ele**: delivery guarantees, sequence numbers, offline queue, presence tracking e fan-out via message broker. Essas são as peças que compõem o Discord, Slack e similares.

3 Arquitetura do Serviço

VISÃO GERAL



COMPONENTES INTERNOS

● Gateway WS

Aceita conexões, autentica via JWT, distribui para canais. Um processo Erlang por conexão.

● Room Manager

Gerencia canais (workspace channels, DMs, broadcast). Persiste metadados e membros no PostgreSQL.

● Presence

Phoenix Presence — CRDT distribuído sem coordenação central. Rastreia quem está online em cada canal.

● Fan-out Engine

Publica mensagens no RabbitMQ exchange. Qualquer sistema integrador assina sua própria fila.

● Offline Queue

Mensagens destinadas a usuários desconectados ficam na fila. Entregues em ordem na reconexão via sequence number.

● Message Store

ScyllaDB particionado por channel_id, clustering por seq. Histórico paginado via cursor — append-only, alta escrita.

4 Protocolo de Mensagens

Mensagens trafegam como **Protobuf sobre WebSocket** — não JSON puro. Isso garante schema versionado, serialização binária eficiente e contratos explícitos entre cliente e servidor.

ESTRUTURA DO FRAME

```
// messages.proto

message ClientFrame {
  uint64   sequence_number = 1; // número de sequência crescente por conexão
  string   channel         = 2; // "workspace:uuid" | "dm:uuid" | "broadcast"
  oneof    payload {
    SendMessage send      = 3;
    AckMessage  ack       = 4;
    TypingEvent typing    = 5;
    PresenceQuery presence = 6;
  }
}

message ServerFrame {
  uint64   sequence_number = 1; // servidor — cliente usa para detectar gap
  string   channel         = 2;
  oneof    payload {
    Message      message = 3;
    Ack          ack      = 4;
    PresenceEvent presence = 5;
    SystemEvent  system   = 6; // heartbeat, reconnect hint
  }
}
```

DELIVERY GUARANTEES

Cada mensagem tem um **sequence_number** crescente por canal. O cliente rastreia o último número recebido. Ao reconectar, envia `resume: { channel, last_sequence_number }` — o servidor entrega as mensagens perdidas em ordem antes de continuar o stream normal. Isso é exatamente o modelo do Discord Gateway v10.

5 Stack e Estrutura do Repositório

Elixir ~1.17

Phoenix Channels

Phoenix Presence

ScyllaDB

Redis

RabbitMQ

Protobuf

gRPC

Meilisearch

MinIO

Svelte

Bun + Elysia

Docker

DECISÃO	ESCOLHA	JUSTIFICATIVA
Runtime	Elixir / Erlang OTP	O BEAM é a arquitetura — presença CRDT, isolamento de conexão e tolerância a falha são propriedades estruturais do runtime, não bibliotecas.
Real-time	Phoenix Channels	Multiplexing nativo, o mesmo modelo que inspirou o Discord Gateway.
Serialização	Protobuf	Schema versionado, binário, contratos explícitos entre cliente e servidor.
Presence	Phoenix Presence	CRDT distribuído — sem coordenação central, consistência eventual.
Histórico de mensagens	ScyllaDB	Wide-column, time-series nativo. Partition key: channel_id, clustering key: seq. Alta escrita, leitura paginada por cursor. Cassandra moderno em C++.
Presença e cache	Redis	TTL natural para eventos efêmeros (typing, last_seen). Pub/sub para notificações de presença entre nós.
Metadados de rooms	ScyllaDB	Modelagem por access pattern: rooms_by_id, members_by_room, rooms_by_user. Sem JOIN, sem serviço extra.
Busca full-text	Meilisearch	Motor de busca open source, self-hostable, escrito em Rust. Indexação async via RabbitMQ consumer. ScyllaDB é o store primário — Meilisearch é só o índice.
Object storage	MinIO	S3-compatible, self-hostable. Armazena arquivos e mídia. Acesso via presigned URLs.
Fan-out	RabbitMQ	Exchange topic para routing genérico por workspace/canal. Integradores assinam filas próprias.
Backend (demo)	Bun + Elysia	REST API que serve o frontend standalone. Existe para demonstração e self-hosting — não faz parte do core do serviço. TypeScript, mesmo ecossistema do frontend.
Frontend (demo)	Svelte	UI standalone para demonstração e portfólio. Consome o core via WebSocket (real-time) e o backend via HTTP (dados). Demonstra o widget em uso.
Widget (distribuível)	Svelte → Web Component	Implementação default de <chat-widget>. Representa o Modelo C de distribuição. O integrador pode ignorar e construir o próprio widget sobre o core JS.

ESTRUTURA DO REPOSITÓRIO

```
chat/
├── core/                                # Elixir – serviço principal
│   ├── lib/chat_core/
│   │   ├── application.ex              # supervision tree
│   │   ├── auth.ex                    # behaviour – integrador fornece implementação
│   │   ├── channels/
│   │   │   ├── user_socket.ex          # handshake, chama Auth.verify/1
│   │   │   ├── room_channel.ex         # canal workspace/DM
│   │   │   └── system_channel.ex       # heartbeat, eventos de sistema
│   │   ├── presence.ex                # Phoenix.Presence CRDT
│   │   ├── rooms/                     # Room Manager (ScyllaDB)
│   │   ├── messages/                  # Message Store (ScyllaDB)
│   │   ├── delivery/
│   │   │   ├── fanout.ex               # publica no RabbitMQ
│   │   │   └── offline_queue.ex        # fila para usuários desconectados
│   │   └── proto/                     # módulos Protobuf gerados
│   ├── priv/migrations/                # schema ScyllaDB
│   ├── config/
│   └── mix.exs
├── backend/                             # Bun + Elysia – REST API (demo/standalone)
│   ├── src/
│   └── package.json
├── frontend/                            # Svelte – UI standalone (demo/portfólio)
│   ├── src/
│   └── package.json
├── widget/                              # Svelte → Web Component (distribuível)
│   ├── src/
│   └── package.json
├── proto/
│   └── messages.proto                  # fonte de verdade: gera tipos Elixir + TypeScript
└── docker-compose.yml                  # profiles: demo | search | storage | local-db | loc
```

Docker Compose profiles — nomes por feature, não por tecnologia:

PROFILE	O QUE SOBE	QUANDO USAR
local-db	ScyllaDB local	Sem instância externa configurada
local-cache	Redis local	Sem instância externa configurada
local-queue	RabbitMQ local	Sem broker externo configurado

PROFILE	O QUE SOBE	QUANDO USAR
search	Motor de busca	Feature de busca habilitada (M7)
storage	Object storage	Feature de mídia habilitada (M5)
demo	backend + frontend + widget	Deployment standalone / portfólio

Se SCYLLADB_URL, REDIS_URL ou RABBITMQ_URL estiverem definidas no env, os profiles local-* são omitidos — o serviço conecta no externo.

6 Contratos de Integração

Por ser um serviço atômico, o chat expõe contratos estáveis que qualquer sistema pode consumir. O que o chat gerencia e o que é responsabilidade do integrador estão explicitamente separados.

RESPONSABILIDADES DO INTEGRADOR

O chat **nunca gerencia**: identidade de usuários, lista de contatos, grafo social, permissões de negócio, dados de perfil. O serviço confia no token recebido e devolve um `user_id` — o que esse ID representa é domínio do integrador.

RESPONSABILIDADE	DE QUEM	COMO
Autenticação	Integrador	Implementa o behaviour <code>Chat.Auth</code> — recebe o token, retorna <code>{:ok, user_id}</code> ou <code>{:error, reason}</code> . JWT é a implementação default.
Object storage	Integrador (opcional)	Implementa o behaviour <code>Chat.Storage</code> — upload, download, delete, <code>presigned_url</code> . MinIO é o default. S3, R2, Oracle Storage ou qualquer provider entram como implementação alternativa.
Message broker	Integrador (opcional)	Configuração via <code>RABBITMQ_URL</code> no env. Se definida, conecta no broker externo do integrador. Se ausente, o profile <code>local-queue</code> sobe uma instância local.
Identidade / perfil	Integrador	O chat conhece só o <code>user_id</code> . Nome, avatar e dados de perfil são fornecidos pelo sistema que integra.
Lista de contatos	Integrador	O chat gerencia membros de sala — quem está em qual room. O grafo social (quem segue quem, lista de amigos) é externo ao serviço.

Sobre o `<chat-widget>`: o projeto fornece uma implementação default como referência. O integrador não é obrigado a usá-la — pode construir seu próprio widget sobre o core JS, com a UI e o comportamento que quiser.

RABBITMQ — EVENTOS PUBLICADOS

EXCHANGE	ROUTING KEY	PAYLOAD
<code>chat.message.inbound</code>	<code>workspace.{id}</code>	<code>workspaceId</code> , <code>userId</code> , <code>text</code> , <code>requestId</code> , <code>receivedAt</code>
<code>chat.presence</code>	<code>workspace.{id}.presence</code>	<code>userId</code> , <code>status</code> (online/offline), <code>channelId</code>

RABBITMQ — EVENTOS CONSUMIDOS

QUEUE	SOURCE	AÇÃO
chat.notifications.inbound	qualquer serviço integrador	Entrega mensagem ao usuário via WebSocket (ou offline queue)

GRPC — API DE ADMINISTRAÇÃO

```
service ChatAdmin {  
  rpc CreateRoom      (CreateRoomRequest) returns (Room);  
  rpc AddMember       (AddMemberRequest)  returns (Member);  
  rpc GetHistory      (HistoryRequest)    returns (stream Message);  
  rpc SendSystemMsg   (SystemMessageRequest) returns (Ack);  
}
```

7 Qualidade e Testes

Meta de cobertura: **80–85%**. Serviços externos (ScyllaDB, Redis, RabbitMQ) rodam via Docker nos testes de integração — sem mock de infraestrutura.

BACKEND — ELIXIR (CORE/)

FERRAMENTA	PROPÓSITO
ExUnit	Framework de testes nativo. Base de tudo.
Phoenix.ChannelCase	Helpers nativos para testar ciclo completo de canal — connect, join, push, assert_reply, assert_broadcast, disconnect.
Mox	Mocks com contrato explícito via behaviours. Usado para dependências externas que não sobem em teste unitário.
ExMachina	Factories para dados de teste.
StreamData	Property-based testing. Sequence numbers, delivery guarantees e edge cases de reconexão têm invariantes que testes unitários comuns não cobrem.
Credo	Análise estática — estilo, code smells, complexidade.
Dialyxir	Type checking via Dialyzer. Pega erros de tipo em tempo de compilação.
ExCoveralls	Relatório de cobertura. Integra com CodeClimate.

FRONTEND E BACKEND JS (FRONTEND/ E BACKEND/)

FERRAMENTA	PROPÓSITO
Vitest	Testes unitários. TypeScript nativo, compatível com Vite.
Svelte Testing Library	Testes de componentes Svelte por comportamento, não implementação.
Playwright	E2E end-to-end conectando em servidor real. Testa o fluxo completo de chat.

CI E QUALIDADE

GitHub Actions — pipeline de CI. **CodeClimate** como gate de qualidade (gratuito para open source) — agrega cobertura, Credo e Dialyxir numa visão unificada. PR bloqueado se cobertura cair abaixo de 80%.

8 CI/CD e Distribuição

Um único workflow (`.github/workflows/ci.yml`) com jobs filtrados por path. Cada job só executa quando seus arquivos relevantes mudam. A dependência do `proto/messages.proto` é explícita — mudanças no protocolo disparam rebuild do `core` e do `widget` garantindo que imagem e pacote publicados estejam sempre em sincronia.

ARTEFATOS PUBLICADOS

ARTEFATO	REGISTRY	TRIGGER
chat-core (imagem Docker)	GHCR — <code>ghcr.io/username/chat-core</code>	Tag v*
@chat/widget (pacote npm)	npm registry	Tag v*

JOBS POR PATH

JOB	DISPARA QUANDO	EM PR	EM TAG V*
core	<code>core/**</code> ou <code>proto/**</code>	test, credo, dialyzer, coverage	+ build image → push GHCR
widget	<code>widget/**</code> ou <code>proto/**</code>	test, build	+ publish npm
backend	<code>backend/**</code>	test, type check	—
frontend	<code>frontend/**</code>	test, build	—

GHCR (GitHub Container Registry) é o registry primário — sem conta separada, sem rate limit em repositórios públicos, integrado ao token do Actions. Mirror opcional para Docker Hub via secret `DOCKERHUB_TOKEN`.

9 Roadmap de Implementação

M0 Fundação

Projeto Phoenix, supervisão OTP, schema ScyllaDB, Docker standalone com todos os serviços, CI básico

M1 Conexão e Autenticação

UserSocket com JWT, Phoenix Channels básico, Protobuf encoding/decoding, heartbeat

M2 Mensagens e Persistência

Send/receive, sequence numbers, Message Store no ScyllaDB, histórico paginado por cursor

M3 Delivery Guarantees

Ack explícito, offline queue, replay na reconexão, at-least-once garantido

M4 Presence e Rooms

Phoenix Presence, room management, typing indicators, online/offline status

M5 Media e Arquivos

Object storage via MinIO, tipo de mensagem file no Protobuf, presigned URLs para acesso, endpoint de upload

M6 Features Estendidas

Threads (parent_id no schema), reações, read receipts, diretório de contatos/usuários

M7 Busca

Meilisearch — indexação async de mensagens e contatos via RabbitMQ consumer, search API

M8 Push Notifications

FCM, APNs e Web Push para usuário totalmente offline — separado da offline queue

M9 Backup e Export

Job periódico de export para object storage (MinIO), histórico completo por workspace